Mid-Decode Model Migration Under a Per-App Memory Limit: Moving a Generation to a Smaller Model Without Holding Both

NagaYu*

Abstract

Prior work first. Closed-form ridge maps between the KV caches of different-size models in one family were introduced by Heo et al. [8]. They apply them at the *prefill hand-off point*. DroidSpeak [9] shares KV caches between models that have the same architecture and are fine-tuned from the same base. This paper adds no new mapping and claims nothing about mapping quality. Its scope is limited to three conditions that this prior work does not address: the switch happens *mid-decode*, between two tokens of one generation; it happens *under memory exhaustion*, triggered by the app's distance to its own per-process limit; and the two models are *never resident together*.

We implement this setting in MLX on Apple silicon and test it against an external jetsam emulator. We compare it with restart, always-small, re-prefill, and the closed-form ridge map of [8] applied mid-decode. That last baseline is reproduced to the letter and runs in the hand-off shape, with both models loaded. For 1.5B→0.5B Qwen2.5 when the app has less spare memory than the 409 MiB a co-resident switch needs, the hand-off shape is killed while every release-first switch survives. But the mapped cache loses the prompt's specifics: needle accuracy is 0.125 with our additions and 0.208 without, against 0.958 for simply re-reading the transcript, which also resumed faster. We state plainly that, in the setting we claim, KV transfer is not the right way to switch. iPhone measurements, energy on device, and human ratings are **PENDING:** not yet collected.

1 Scope and prior work

What is not ours. Mapping one model's KV cache into another model's geometry is prior work. Heo et al. [8] fit, for every target layer and KV head, separate key and value maps in closed form by ridge regression ($\lambda = 0.01$). The input is the concatenated KV heads of the top-k source layers, ranked by single-source R^2. Keys have RoPE [12] removed before the map and re-applied after it. The transfer happens at the *prefill hand-off point*: the receiver gets the sender's prefix cache and decodes from there. Their main results are small-to-large over matched KV geometry. Large-to-small is reported only on HellaSwag, where one pair keeps 37% of standalone accuracy. DroidSpeak [9] reuses KV caches between models that share an architecture and a foundation model. It recomputes selected layers and reuses the rest, at prefill time. Re-reading a transcript with a new model (re-prefill) is a standard alternative, and we treat it as a baseline. Keeping the first tokens because attention concentrates on them follows StreamingLLM [13].

*Code, mappers and raw measurements: `https://github.com/NagaYu/molt`. Draft generated October 18, 2026; device results pending (see Section 7).

What this paper claims, and only this. We study one situation. An on-device app is in the middle of generating an answer with a model that nearly fills the app's memory budget. Then something else in the app needs memory. On iOS the app is killed once it crosses its per-process limit [2]. The goal is to continue *the same sentence* with a smaller model, with no retracted tokens and no termination. The contribution is restricted to three conditions:

1. **Mid-decode.** The switch happens between two tokens of one generation. The receiver takes the sender's cache for every token fed so far and itself feeds the token already shown to the user.
2. **Under memory exhaustion.** The switch is triggered by the app's headroom to its own limit (`os_proc_available_memory` [4]), polled before every token, before the limit is crossed.
3. **Non-co-resident.** Sender and receiver weights are never resident together. We show what this requires on MLX [7] and test that the kernel-tracked peak footprint stays under the cap.

Nothing else in this paper is claimed as new. That includes the mapping, its fitting procedure, RoPE handling, re-prefill and sink tokens.

The question that decides whether there is a claim at all. If the simple method is enough in this setting, molt has nothing to claim. The simple method is the closed-form map of [8] applied at the token boundary. We therefore reproduce it without shortcuts (condition C) and measure where it is and is not enough. Section 6 states plainly where our own hypothesis fails.

2 The setting

The limit an app actually faces. Apple documents `os_proc_available_memory()` as "the current memory limit minus the memory footprint of your app". It returns 0 once the limit is exceeded. The limit can change during the app's life cycle, and the value is advisory and should not be cached [4]. A jetsam event report records the reason for a termination. `per-process-limit` means the app crossed the limit imposed on all apps; `vm-pageshortage` means system-wide pressure [2]. The UIKit memory warning [3] and the dispatch memory-pressure source [5] report *system* memory pressure. We found no documentation that promises either one fires before a `per-process-limit` termination. Some devices honour a larger limit under the `increased-memory-limit` entitlement [1]. The quantity the limit is checked against is the physical footprint. On Apple silicon, MLX tensors sit in Metal buffers that the resident set size barely counts. We measured a 1 GiB MLX array moving RSS by ≈0 MiB and the footprint by ≈1 GiB. Every memory number in this paper is therefore phys_footprint, never RSS.

The event. A generation runs on model S, whose footprint leaves headroom h under the limit. At token n_p, the app's other work starts to need B more memory, arriving in chunks of 64 MiB per token. B exceeds h, so S cannot stay, while the smaller model T plus B fits. A condition survives if its footprint never exceeds the limit. A co-resident switch needs about T's weights plus the mapper in spare memory at the moment it starts. A non-co-resident switch needs only the KV caches and the mapper. The interesting band is

$$\underbrace{\text{chunk} + \text{margin}}_{\text{ordered needs}} \;\le\; h \;<\; \underbrace{w_T + w_{\text{map}} + \text{margin}}_{\text{co-resident needs}},$$

where one schedule survives and the other cannot. Section 5.4 measures whether the band exists and how wide it is for real tiers.

3 Method

3.1 The mapping (prior work, condition C)

We reproduce the recipe of [8] as written.

- *Features.* Per source layer, all KV heads are flattened. Keys have the source RoPE removed; values are used as is.
- *Targets.* The destination's own keys (RoPE removed) and values, per layer and head.
- *Regression.* Centred ridge with $\lambda = 0.01$ and bias $b = \bar{y} - \bar{x}W$. Covariance is accumulated in float32 and solved in float64.
- *Data.* 500 FineWeb-Edu [10] sequences of 1 024 tokens, sub-sampled with stride 4.
- *Source selection.* For each destination layer, the top-k source layers are kept by a fixed ranking: head-averaged single-source R^2, averaged over K and V.

Keys are re-rotated with the destination RoPE at the same absolute positions.

We had to make two interpretations, and we state both. (i) The text writes $X^\top X + \lambda I$ without saying whether $X^\top X$ is normalised. On ∼128 K rows an unnormalised Gram matrix would make $\lambda = 0.01$ a no-op, and their reported collapse at $\lambda = 1$ could not happen. We therefore read $X^\top X$ as the centred covariance and also sweep λ (Table 2). (ii) Heo et al. choose k by downstream accuracy. We choose k by held-out prefix-KV negative log-likelihood (NLL), measured on documents disjoint from both the calibration and the evaluation splits.

Implementation checks in `fit.selftest` and `migrate.selftest`: a planted linear map is recovered to $R^2 = 1$; the R^2 computed from sufficient statistics equals the R^2 of actually applying the fitted mapper to within 10^{-7}; and with an identity mapper, transplanting reproduces a native re-prefill to within 1.4×10^{-6} on the test models. On the real 4-bit Qwen2.5-0.5B, mapping its own cache onto itself through the full RoPE strip-and-reapply path changes the logits by at most 0.110 (fp16 rounding). At experiment Q's switch point it gives the same 8-token greedy continuation as native decoding on 24 of 24 needle items.

3.2 Mid-decode transplant

The decode loop keeps one invariant: the cache holds KV for exactly the tokens fed so far, and `pending` is the token already shown to the user but not yet fed. At a switch the receiver gets the mapped cache and feeds `pending` itself, so no shown token is retracted. Molt adds two optional *anchors*, which we treat as engineering choices and not as contributions.

- The first s positions are replaced by the receiver's native KV. This is exact, because causal positions depend only on earlier tokens; it follows the sink observation of [13].
- The last t positions are re-fed natively over the mapped history.

Setting $t =$ all gives re-prefill (condition A′). s and $t \leq 32$ are chosen on held-out text only.

3.3 The memory order (the claim)

Releasing a model on MLX is not one call. We measured three facts (Appendix A). (1) After `del`, buffers stay in MLX's allocator cache and are not reused for arrays of other sizes. (2) `mx.clear_cache()` right after `del` can free nothing, because a buffer returns to the cache only after the GPU has finished the last command buffer that used it. (3) After synchronising and clearing, the OS still reclaims the memory asynchronously, in about 10^2 ms. The *ordered* schedule therefore runs as follows:

1. snapshot the source KV;
2. drop the source weights, synchronise, clear, and poll phys_footprint until it has fallen by 90% of the source weight bytes;
3. load the mapper and map;
4. drop the mapper and source KV the same way;
5. load the receiver;
6. apply the anchors and resume.

Whether steps (2) and (4) are needed at all turns out to depend on MLX's cache limit (Section 5.4). Table 5 lists the three alternatives we measure against it. *colocated* is the prefill hand-off shape. *del-only* is the natural "drop it, then load". *clear-no-wait* is the call a developer would write.

The trigger. Before every token the loop reads the headroom. On iOS that is the return value of `os_proc_available_memory()`; on the Mac it is cap − phys_footprint. It migrates when the headroom falls below what *the chosen schedule* needs plus the next demand chunk. Each schedule thus gets its own best trigger, and a co-resident schedule is never killed merely for reacting late.

4 Experimental setup

Models. We use Qwen2.5 Instruct [11] in the 4-bit MLX builds an app would ship. The pairs, large to small (the direction memory pressure forces), are 3B→1.5B (matched KV geometry: 2 KV heads, $d_h = 128$) and 1.5B→0.5B (mismatched: d_h 128→64). Mappers are fitted on these exact 4-bit weights. All three sizes share one tokenizer, which we checked by vocabulary hash. Decoding is greedy, so conditions differ only in what the switch did.

Conditions. All conditions run on one machine. Each run is a separate process, so no condition inherits another's allocator state.

- **A** restart: discard the generation and start over on the small model.
- **A′** re-prefill: release, load small, and re-read the prompt plus everything shown so far. This baseline is added because it is the obvious alternative; it is not one of the four named in the brief.
- **B** always small.
- **C** the closed-form map of [8] applied at the token boundary, with the selected k^*, in hand-off shape (colocated) and without anchors.

- **D** molt: the same mapper, the ordered schedule and the held-out-selected anchors.

Ablations change one factor each: C+ordered isolates the schedule, D+colocated the anchors, D with the depth-linear mapper the mapper. D with del-only and D with clear-no-wait test the naive freeing variants.

Jetsam emulation. A separate C process polls the worker's `proc_pid_rusage` at about 1 kHz. It records the instantaneous phys_footprint and the kernel ledger's lifetime maximum (`ri_lifetime_max_phys_footprint`), and sends `SIGKILL` the moment either exceeds the cap. The lifetime maximum cannot miss a spike between samples. Phase labels written by the worker attribute every sample to a phase.

Experiments. *I (intrinsic)*: teacher-forced NLL of 128 held-out tokens after the switch, on the FineWeb-Edu evaluation split, at boundary positions $P \in \{32, 128, 256, 512, 768\}$. *Q (quality)*: no cap, with the switch at a fixed token per item. The suite has GSM8K [6] (switch at token 32, mid-reasoning), a needle task with 14 similar records (switch right before the answer value), and open-ended explanations (switch at token 48). *M (memory)*: a cap of $S+h$, where S is the measured footprint of the big model decoding, swept over h; demand arrives as in Section 2.

Metrics.

- **Survival**: the run was not killed.
- **Lost tokens**: tokens shown and then retracted.
- **Visible seam**: the jump in judge NLL across the boundary, $\overline{\mathrm{NLL}}[b, b{+}16) - \overline{\mathrm{NLL}}[b{-}16, b)$, reported relative to A′; and human ratings, which are pending.
- **Continuation quality**: task accuracy, with a killed run scoring 0 and never dropped, plus judge top-1 agreement after the boundary. The judge is the pair's large model.
- **Time to resume**: from trigger to the next shown token. For A, the time until new content appears.
- **Peak footprint during migration**: kernel lifetime maximum.
- **Energy**: CPU energy per process (`ri_energy_nj`) and GPU energy system-wide from IOReport minus an idle baseline (Appendix B).

Throughput is recorded but is not a primary metric. Every aggregate carries a 95% bootstrap interval, paired by item where two conditions are compared.

Platform. Reported per run in the result files: Apple M2, 16 GB, macOS 27.0.1, MLX 0.31.2, mlx-lm 0.31.3. Other users' processes were running during the measurements and swap was nearly full (Section 7). Timings are reported but should be read with that in mind; footprint and survival are not affected.

5 Results

5.1 The prior mapping, reproduced

Figure 1 and Table 1 cover every fitted mapper.

Table 1: Every fitted mapper. Held-out = documents disjoint from calibration and evaluation.

pair	mapper	k	params (M)	MiB	R^2 in	R^2_K held	R^2_V held	NLL held
1.5B→0.5B	depth_linear	1	1.6	3	0.592	0.702	0.468	4.285
1.5B→0.5B	heo_k1	1	1.6	3	0.616	0.720	0.497	3.326
1.5B→0.5B	heo_k2	2	3.2	6	0.682	0.769	0.578	3.096
1.5B→0.5B	heo_k4	4	6.3	12	0.723	0.797	0.629	2.996
1.5B→0.5B	heo_k8	8	12.6	24	0.760	0.820	0.671	2.948
1.5B→0.5B	heo_k16	16	25.2	48	0.787	0.834	0.697	2.923
1.5B→0.5B	heo_kall	28	44.0	84	0.806	0.842	0.708	2.937

Table 2: Ridge λ sweep at the selected k^* (centred-covariance reading).

pair	λ	held-out NLL [95% CI]
1.5B→0.5B	0.0	2.864 [2.719, 3.010]
1.5B→0.5B	0.001	2.865 [2.720, 3.010]
1.5B→0.5B	0.01	2.869 [2.724, 3.014]
1.5B→0.5B	0.1	2.897 [2.750, 3.041]
1.5B→0.5B	1.0	3.144 [2.988, 3.296]

1.5B→0.5B, mismatched KV geometry. Heo et al. list this case as future work: head dimension 128→64, 28→24 layers. Calibration used 128,000 rows. On 64 held-out documents, prefix-KV NLL improves monotonically from $k = 1$ (3.326) to $k = 16$ (2.923). At $k =$ all 28 layers it is slightly worse (2.937), so $k^* = 16$ (48 MiB in fp16). For reference, the receiver reading the same prefix itself scores 2.800 and the sender 2.448. The transferred cache therefore costs 0.123 nats per token *more* than letting the small model re-read the text. Held-out R^2 at k^* is 0.834 for keys and 0.697 for values. Values are consistently the harder target.

molt v0.1's own layer pairing (depth-linear, one source layer per target layer) has nearly the same R^2 as Heo's $k = 1$. That is 0.702/0.468 for depth-linear against 0.720/0.497 for $k = 1$. Its NLL, however, is far worse: 4.285 against 3.326. *Which* layers feed a target matters more than the per-layer R^2 suggests, and the prior method's R^2 ranking gets it right where v0.1 did not.

λ and anchors (on a 32-document subset of the held-out split, so absolute values differ from the 64-document sweep). NLL is flat from $\lambda = 0$ (2.864) through $\lambda = 0.01$ (2.869). It degrades at 0.1 (2.897) and collapses at 1 (3.144) (Table 2). This is the shape Heo et al. describe for their own sweep, which supports reading $X^\top X$ as a covariance. Native anchors help, but only a little. With no anchors the NLL is 2.869; the best setting within the 32-token tail budget is $s = 4$, $t = 32$ at 2.844. A 128-token tail reaches 2.813, and re-reading everything reaches 2.761. On this intrinsic measure no partial-native setting closes the gap to re-prefill.

5.2 Transfer cost against switch position

1.5B→0.5B. Teacher-forced, the transferred cache is never as good as letting the receiver read the text itself, and the gap grows with context (Figure 2). For C the gap is +0.023 nats per token at $P = 32$ and +0.146 at $P = 768$, CI [+0.111, +0.196]. molt's anchors narrow it to +0.117 at $P = 768$. At $P = 32$ the 32-token native tail covers the whole context, so D *is* a re-prefill there (+0.000). Per token these gaps look small: about a tenth of a nat, against the 2.448→2.800 difference between the two models. Section 5.3 shows they are not small once the receiver generates

Table 3: Experiment Q. Accuracy counts every item; jump = judge-NLL jump at the switch (16 tokens each side).

pair	condition	GSM8K	needle	agree$_{post}$	jump	lost	resume (s)
1.5B→0.5B	big (no pressure)	0.45	1.00	0.998	–	0	–
1.5B→0.5B	A restart	0.00	0.96	0.860	–	29	1.76
1.5B→0.5B	A′ re-prefill	0.25	0.96	0.858	+0.352	0	1.33
1.5B→0.5B	B always small	0.00	0.96	0.860	–	0	–
1.5B→0.5B	C ridge mid-decode	0.00	0.21	0.686	+1.779	0	1.64
1.5B→0.5B	D molt	0.05	0.12	0.751	+1.441	0	2.15
1.5B→0.5B	C + ordered	0.00	0.21	0.686	+1.779	0	1.43
1.5B→0.5B	D, depth-linear	0.05	0.00	0.720	+3.107	0	1.52

freely.

5.3 Quality when the switch lands mid-decode

1.5B→0.5B: the transferred cache loses the prompt's specifics. Every migrating condition switches at the same token (Figure 3 and Table 3).

- **Needle** (the switch lands just before the value is written). Re-prefill A′ answers 0.958 of the items correctly, the same as the small model alone (B, 0.958). C answers 0.208 and molt (D) 0.125. The failures are near-misses, such as "8563" for 8564, or plausible wrong codes: the receiver knows it must write a four-digit code, but not which one.
- **GSM8K** (the switch lands mid-reasoning). The big model alone scores 0.450. A′ scores 0.250, *above* B (0.000), because the big model's first 32 tokens set up a short, correct derivation that the small model continues. C scores 0.000 and D 0.050. Their continuations stay fluent and on-topic but bring in quantities the problem never states. Every C and D run hits the 256-token cap without finishing (20/20 and 20/20, against 12/20 for A′).
- **Judge agreement after the switch** (all 68 items). A′ 0.858, C 0.686, D 0.751. The seam jump in judge NLL at the boundary is 0.352 for A′, 1.779 for C and 1.441 for D.

Paired by item:

- D improves on C. Agreement +0.065, CI [0.035, 0.096]; seam -0.338, CI [-0.561, -0.118]. Task score is unchanged (-0.023, CI [-0.114, 0.045]).
- D is far below A′. Task -0.545, CI [-0.705, -0.364]; agreement -0.107, CI [-0.151, -0.066]; seam +1.089, CI [0.777, 1.422].

A restart (A) produces exactly B's text, because decoding is greedy, and before that it retracts every token already shown: 8, 32 or 48 depending on the task, 29.2 on average.

5.4 Survival under a per-process limit

1.5B→0.5B. The big model decodes at $S = 1380$ MiB and the small one at 763 MiB. A co-resident switch needs ≈409 MiB of headroom: 265 MiB of receiver weights, the 48 MiB mapper and a margin. We swept $h \in \{128, 205, 345, 569, 818\}$ MiB, with 3 runs per cell (Figure 4 and Table 4). Doing nothing (*big, ignores pressure*) is killed in every cell, so the emulator does reproduce the

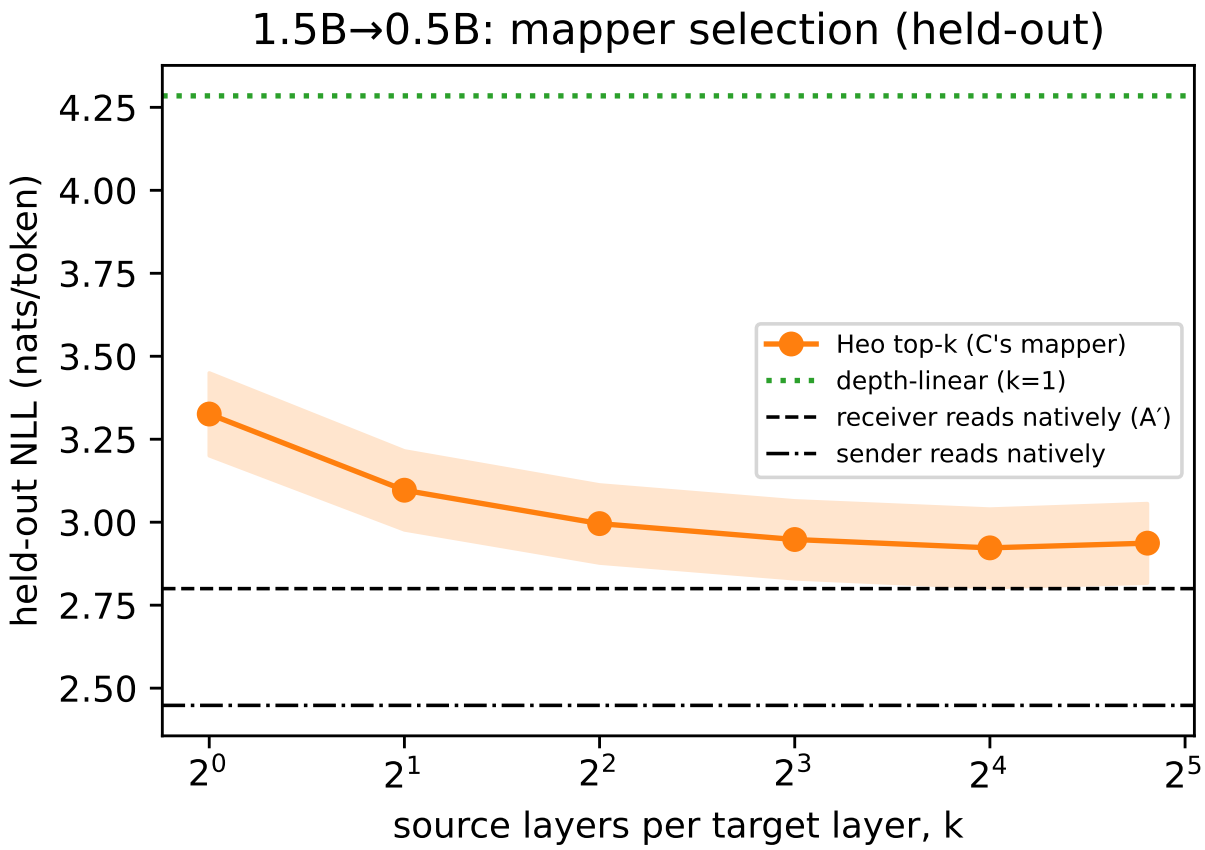


Figure 1: Held-out prefix-KV NLL against the number of source layers k used by the closed-form mapper of [8], with 95% CIs. Dashed: the receiver reading the same prefix natively (what re-prefill achieves). Dash-dot: the sender reading it natively.

kill. The hand-off shape is killed in `load_dst` or `transplant` whenever h is below its need: C is 0/3 at $h = 128$ and 0/3 at $h = 345$. It survives above the need: 3/3 at $h = 569$. Every schedule that releases the sender first survives every cell: A, A′, D and C+ordered. At $h = 345$ the kernel lifetime-maximum footprints are 1671 MiB for D and 1721 MiB for A′, under a cap of 1725 MiB. Figure 5 shows the footprint through the switch.

The naive freeing variants depend on the allocator. Under MLX's default buffer cache, D with del-only is killed in every cell, including $h = 818$ (0/3). D with clear-no-wait is erratic: 0/3, 3/3, 0/3, 2/3 and 0/3 across the grid. This matches Appendix A: `clear_cache` only sometimes frees. With the 20 MiB cache limit that mlx-swift's examples set on iOS, both variants survive every cell tested. For example, at $h = 345$ del-only is 3/3 and clear-no-wait 3/3. C is still 0/3.

Time to resume and energy at $h = 345$. A′ resumes in 0.48 s, D in 0.79 s and C+ordered in 0.66 s. Releasing the sender costs about 0.298 s in every condition. That time goes to garbage collection, synchronisation and unmapping; the wait for the footprint is ≈0. D additionally pays 0.280 s in its `map` phase, most of it the second release (mapper and sender KV). CPU energy for the switch is 0.83 J for A′ and 1.30 J for D. GPU energy could not be attributed: other processes drew 12.0 W of GPU power in the same runs. With models resident, mapping is cheaper than re-prefill from 256 tokens of context up (Figure 6). At 1 024 tokens it takes 0.196 s against 0.647 s, and at 4 096 tokens 1.419 s against 2.669 s. In our runs the context at the switch is short: median 89 tokens for

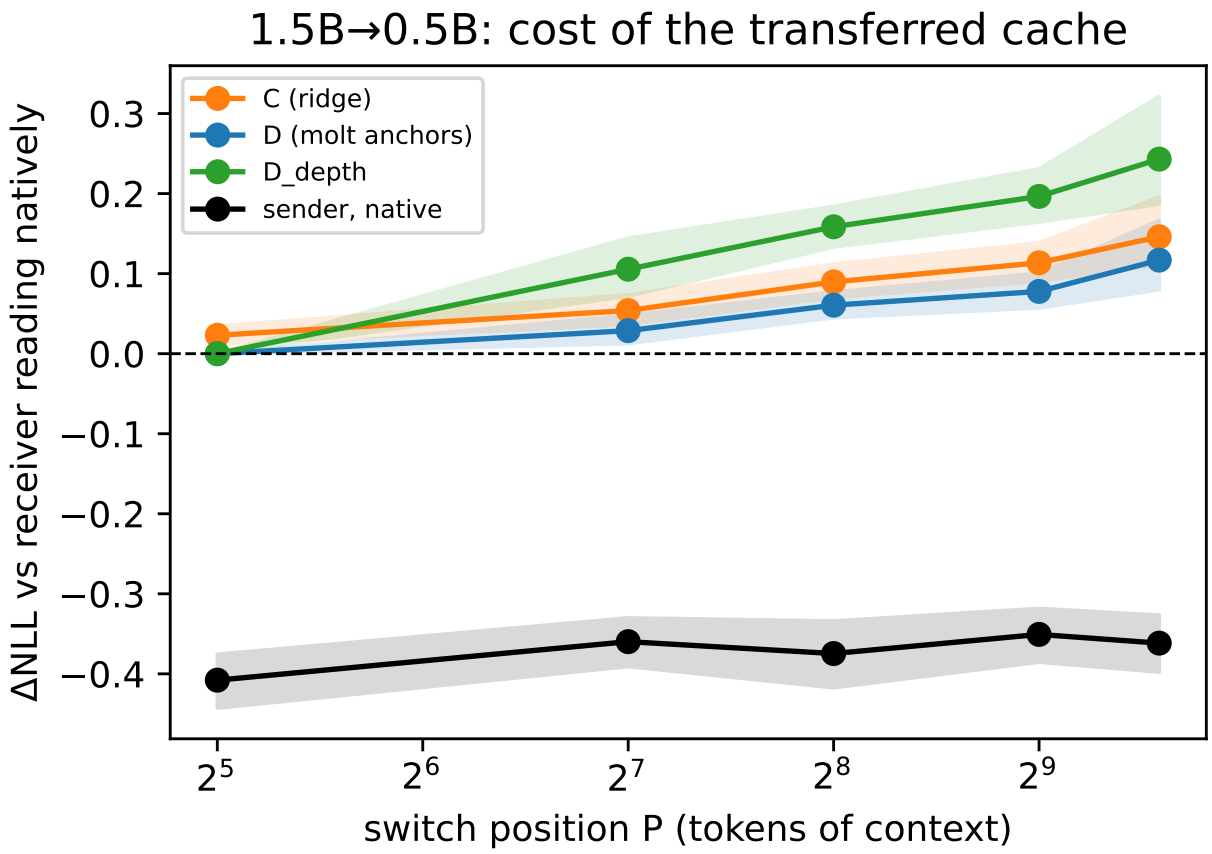


Figure 2: Experiment I. Extra NLL of the receiver decoding from a transferred cache, relative to reading the same text natively, as a function of how much context precedes the switch. Bands are paired 95% CIs over evaluation documents.

open prompts, 147 for GSM8K and 298 for the needle task. There, the mapping's compute saving is smaller than the extra release it requires.

6 Where the hypothesis fails, stated plainly

Where the hypothesis fails, stated plainly. The design hypothesis had two parts. First, carrying the KV cache across the switch would let the small model continue *the same sentence* better than discarding or re-reading the context. Second, molt's additions would make the simple method of [8] sufficient where it alone is not. For 1.5B→0.5B, it does not. The 3B→1.5B pair is **PENDING:** still running.

The transferred cache does not carry the content. With a mapped cache (C or D), the receiver keeps the topic and the voice of the text but loses its specifics. Needle accuracy is 0.208 for C and 0.125 for D, against 0.958 when the small model re-reads the text. On GSM8K the scores are 0.000 and 0.050, against 0.250. molt's anchors make the seam smaller than C's. Paired agreement is +0.065, CI [0.035, 0.096], and the seam is -0.338 nats, CI [-0.561, -0.118]. Task score does not move: -0.023, CI [-0.114, 0.045]. "Better than C" and "good enough" are different claims,

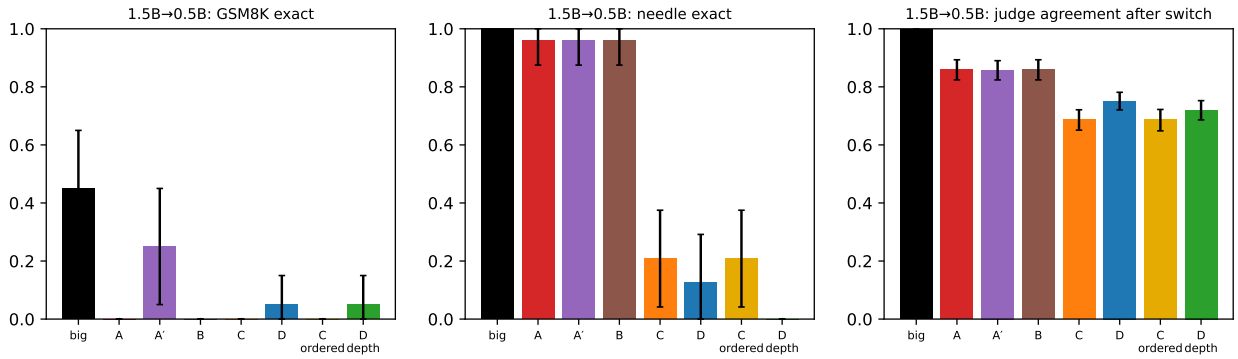


Figure 3: Experiment Q. GSM8K and needle exact match, and judge top-1 agreement after the switch, per condition. Error bars are 95% bootstrap CIs.

and only the first one holds here. This is not an implementation fault. The same code path with an identity map on one model reproduces native decoding on 24 of 24 needle items (Section 3.1), so the loss comes from the cross-model map. The teacher-forced gap of about a tenth of a nat per token (Section 5.2) understates this badly: it compounds once the receiver generates freely.

Re-prefill is the better switch in exactly the setting we claim. A′ meets all three conditions of our scope. It switches mid-decode with no retracted tokens, it is triggered by headroom, and it releases the sender before loading the receiver. It survives every cell in which C is killed. In our runs it resumes faster than D (0.48 s against 0.79 s at $h = 345$) and uses less CPU energy (0.83 J against 1.30 J). The mapping is cheaper only in compute and only with longer context: 1.419 s against 2.669 s at 4 096 tokens. Our quality runs switched at 89–298 tokens of context (medians per task), and the intrinsic gap grows with context (+0.117 at 768 tokens). So a long-context advantage for the transferred cache is *untested*, and on the trend we measured, unlikely.

The careful release is a fix for one allocator setting, not a general requirement. Under MLX's default buffer cache, drop-then-load is killed in every cell, and the plain `clear_cache()` variant survives erratically. There, synchronise-clear-wait is what makes the ordered schedule survive. Under the 20 MiB cache limit that mlx-swift's examples set on iOS, both naive variants survive every cell we ran. What holds under both settings is only the *order*: the receiver must not be loaded before the sender is released. That is a necessary condition, not a technique.

Nothing of molt v0.1's mapper survives. Its depth-linear layer pairing is far worse than the prior method's R^2-ranked top-k. Held-out NLL is 4.285 against 2.923; needle accuracy is 0.000.

What does hold. Below the headroom a co-resident switch needs (409 MiB here), the prefill-hand-off shape is killed. A release-first switch completes with no retracted token, under both allocator settings. At the context lengths we measured, the best thing to give the receiver in that situation is the transcript, not the transferred KV.

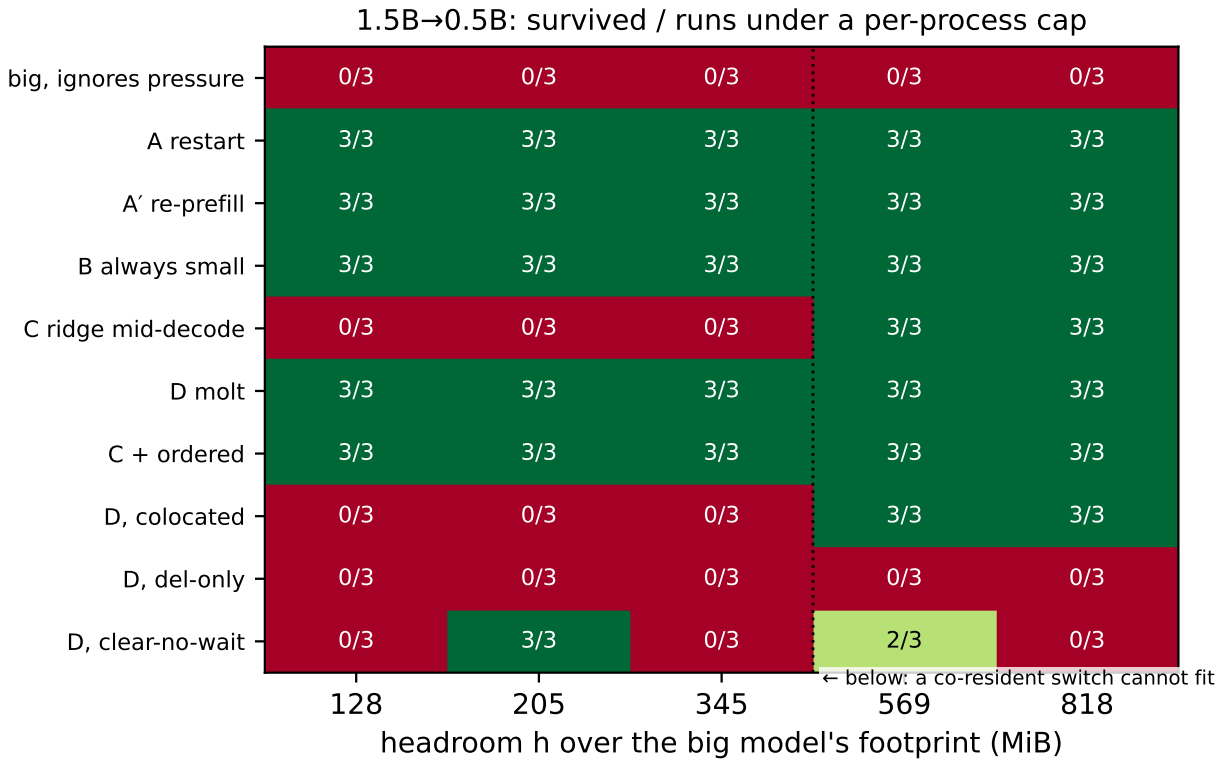


Figure 4: Experiment M. Survival rate against headroom h over the big model's own footprint, one process per run under the jetsam emulator. The dotted line marks the headroom a co-resident switch needs (receiver weights + mapper + margin).

7 Limitations

- **No iPhone results yet.** Every number in this draft comes from an M2 Mac with a jetsam *emulator*. The device protocol is written and its platform core is self-tested on macOS (`ios/`). It covers the Memory Budget Test that reproduces real kills, Instruments traces, and at least three device generations. The app itself has not been compiled or run, because the build machine's Xcode license is not accepted and no device is attached. The simulator is not used: it enforces no per-app limit. Real limits, the `increased-memory-limit` entitlement, and iOS's own reclaim timing could all move the band of Section 5.4.
- **A shared, overloaded machine.** Other users' processes kept swap nearly full throughout. Footprint, survival and quality are unaffected. Times and GPU energy are inflated and noisy, and are reported with the machine's load average.
- **Energy is partial.** CPU energy is per process. GPU energy is system-wide minus an idle baseline. DRAM energy is missing. A `powermetrics` cross-check is pending.
- **Human ratings are pending.** The seam is measured only with a model judge, which is the pair's own large model.
- **Scope.** One model family (Qwen2.5), 4-bit weights, greedy decoding, English text.

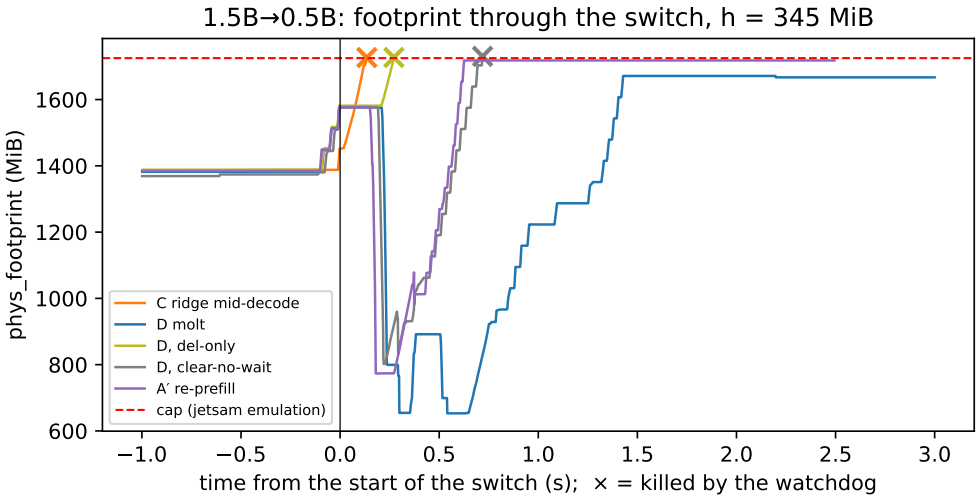


Figure 5: Experiment M, 1.5B→0.5B at $h = 345$ MiB (inside the band). phys_footprint sampled at ∼1 kHz by the external watchdog, aligned at the start of the switch; × marks the watchdog's kill.

8 Conclusion

We moved a generation from a larger to a smaller model mid-decode, triggered by the app's headroom, without ever holding both. Against a faithful reproduction of the closed-form KV map of [8], the measurements give one result that holds and two that do not.

- **Holds.** The memory order decides survival. Loading the receiver before releasing the sender, as a prefill hand-off does, is killed whenever headroom is below the receiver plus mapper. Releasing first survives.
- **Does not hold.** Mapping the cache did not carry the prompt's content into the small model, with or without our anchors.
- **Does not hold.** Re-reading the transcript was better on every quality measure, and also faster at the context lengths we ran.

For an on-device app facing a per-process limit mid-answer, our data recommend a release-first re-prefill, not KV transfer. Device measurements on iPhones (Section 7) are the next step, because iOS's own limits and reclaim behaviour could move the band.

References

[1] Apple Inc. com.apple.developer.kernel.increased-memory-limit. Apple Developer Documentation, `https://developer.apple.com/documentation/bundleresources/entitlements/com.apple.developer.kernel.increased-memory-limit`, . Accessed 2026-10-17.

[2] Apple Inc. Identifying high-memory use with jetsam event reports. Apple Developer Documentation, `https://developer.apple.com/documentation/xcode/identifying-high-memory-use-with-jetsam-event-reports`, . Accessed 2026-10-17.

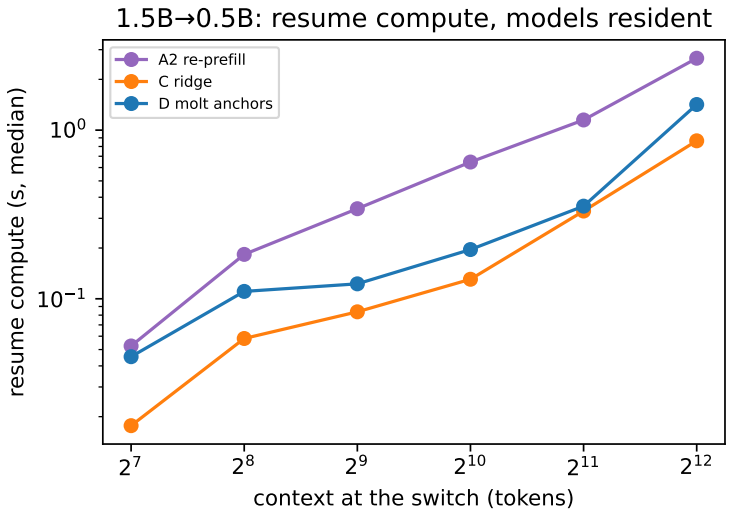


Figure 6: Experiment R. Compute to resume as a function of context at the switch, with both models resident (load costs excluded; median of 5 interleaved repeats).

[3] Apple Inc. didreceivememorywarningnotification. Apple Developer Documentation, `https://developer.apple.com/documentation/uikit/uiapplication/didreceivememorywarningnotification`, . Accessed 2026-10-17.

[4] Apple Inc. os_proc_available_memory. Apple Developer Documentation, `https://developer.apple.com/documentation/os/os_proc_available_memory`, . Accessed 2026-10-17.

[5] Apple Inc. makememorypressuresource(eventmask:queue:). Apple Developer Documentation, `https://developer.apple.com/documentation/dispatch/dispatchsource/makememorypressuresource(eventmask:queue:)`, . Accessed 2026-10-17.

[6] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[7] Awni Hannun, Jagrit Digani, Angelos Katharopoulos, and Ronan Collobert. MLX: Efficient and flexible machine learning on Apple silicon. `https://github.com/ml-explore/mlx`, 2023.

[8] Taekyung Heo, Rasoul Shafipour, Ritchie Zhao, Maximilian Golub, Mohammad Mahdi Kamani, Ritika Borkar, Makesh Tarun Chandran, Pantea Zardoshti, and Bita Darvish Rouhani. Cross-model KV cache transfer in LLM families: A closed-form linear mapping for prefill reuse. *arXiv preprint arXiv:2608.03893*, 2026.

[9] Yuhan Liu, Yuyang Huang, Jiayi Yao, Shaoting Feng, Zhuohan Gu, Kuntai Du, Hanchen Li, Yihua Cheng, Junchen Jiang, Shan Lu, Madan Musuvathi, and Esha Choukse. DroidSpeak: KV cache sharing for cross-LLM communication and multi-LLM serving. *arXiv preprint arXiv:2411.02820*, 2024.

[10] Guilherme Penedo, Hynek Kydlíček, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. The FineWeb datasets: Decanting the web for the finest text data at scale. *arXiv preprint arXiv:2406.17557*, 2024.

[11] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[12] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.

[13] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2023.

A What freeing a model costs on MLX

All three facts below are measured by `mlx/molt_mlx/footprint.py::selftest` on every run of the test suite.

1. A 256 MiB MLX array adds 262 MiB to phys_footprint, while total RSS is 32 MiB.
2. After `del` the footprint is unchanged (+262 MiB). `mx.clear_cache()` immediately after `del`, followed by a 300 ms sleep, freed the buffer in only 0 of 5 runs; in the others the footprint stayed up by as much as +262 MiB. Whether it frees depends on whether the GPU has finished with the buffer, so it is not something to rely on.
3. `mx.synchronize()`, then `clear_cache()`, then polling returned the memory every time, with a median of 175 ms (range 40–185 ms).

The peak-footprint test (`mlx/tests/test_peak_footprint.py`) runs each schedule in its own process under a cap of (source-resident footprint + 40 MiB). The models are a synthetic 290 MiB source and a 109 MiB destination, so the cap is below source plus destination. Ordered peaks at 694 MiB under a 731 MiB cap; colocated and del-only are killed in `load_dst`. clear-no-wait survived 3 of 3 runs at this size, so it is *not* shown to fail there; the full-size result is in Section 5.4.

B Energy without root

`ri_energy_nj` counts CPU energy only: a busy MLX matmul loop reads 0.02 W in it. IOReport's "GPU Energy" channel reads 11.58 W for the same loop, with no root required. Energy is reported as these two named parts and never summed silently. DRAM energy is unavailable without `powermetrics`; `bench/powermetrics.sh` is the root cross-check.

C Reproducing every number

```
python -m pytest mlx/tests --import-mode=importlib  # selftests + peak test
python -m bench.facts                      # MLX freeing + energy facts
python -m bench.fit_mappers --pair 1.5b-0.5b   # fit, sweep, select
python -m bench.intrinsic   --pair 1.5b-0.5b   # experiment I
python -m bench.run_memory  --pair 1.5b-0.5b   # experiment M
python -m bench.run_quality --pair 1.5b-0.5b   # experiment Q
python -m bench.score       --pair 1.5b-0.5b   # judge scores
```

```
python -m bench.analyze                    # bench/results/summary.json
python figures/make_paper_figures.py       # figures + this paper's macros
python -m bench.make_results_md            # RESULTS.md
cd ios && swift run MoltCoreSelfTest       # Swift core
```

`python -m bench.run_all` runs the middle block for both pairs, resumably. Figures and every number in the text are produced from `bench/results`; none is typed by hand.

Table 4: Experiment M: every cell. Peak = kernel lifetime-max phys_footprint (MiB).

pair	h	cap	condition	survived	peak	killed in	resume (s)
1.5B→0.5B	128	1508	A restart	3/3	1445	–	0.46
1.5B→0.5B	128	1508	A′ re-prefill	3/3	1465	–	0.49
1.5B→0.5B	128	1508	B always small	3/3	1419	–	–
1.5B→0.5B	128	1508	C ridge mid-decode	0/3	1510	load_dst	–
1.5B→0.5B	128	1508	C + ordered	3/3	1449	–	0.88
1.5B→0.5B	128	1508	D molt	3/3	1450	–	0.87
1.5B→0.5B	128	1508	D, clear-no-wait	0/3	1514	decode_dst	–
1.5B→0.5B	128	1508	D, colocated	0/3	1512	load_dst	–
1.5B→0.5B	128	1508	D, del-only	0/3	1513	load_dst	–
1.5B→0.5B	128	1508	big, ignores pressure	0/3	1510	decode_src	–
1.5B→0.5B	205	1585	A restart	3/3	1494	–	0.53
1.5B→0.5B	205	1585	A′ re-prefill	3/3	1529	–	0.56
1.5B→0.5B	205	1585	B always small	3/3	1487	–	–
1.5B→0.5B	205	1585	C ridge mid-decode	0/3	1588	load_dst	–
1.5B→0.5B	205	1585	C + ordered	3/3	1450	–	0.83
1.5B→0.5B	205	1585	D molt	3/3	1484	–	0.84
1.5B→0.5B	205	1585	D, clear-no-wait	3/3	1573	–	0.40
1.5B→0.5B	205	1585	D, colocated	0/3	1589	load_dst	–
1.5B→0.5B	205	1585	D, del-only	0/3	1588	load_dst	–
1.5B→0.5B	205	1585	big, ignores pressure	0/3	1590	decode_src	–
1.5B→0.5B	345	1725	A restart	3/3	1681	–	0.51
1.5B→0.5B	345	1725	A′ re-prefill	3/3	1721	–	0.48
1.5B→0.5B	345	1725	B always small	3/3	1676	–	–
1.5B→0.5B	345	1725	C ridge mid-decode	0/3	1728	transplant	–
1.5B→0.5B	345	1725	C + ordered	3/3	1637	–	0.66
1.5B→0.5B	345	1725	D molt	3/3	1671	–	0.79
1.5B→0.5B	345	1725	D, clear-no-wait	0/3	1731	decode_dst	–
1.5B→0.5B	345	1725	D, colocated	0/3	1735	transplant	–
1.5B→0.5B	345	1725	D, del-only	0/3	1727	load_dst	–
1.5B→0.5B	345	1725	big, ignores pressure	0/3	1734	decode_src	–
1.5B→0.5B	569	1949	A restart	3/3	1871	–	0.79
1.5B→0.5B	569	1949	A′ re-prefill	3/3	1916	–	0.58
1.5B→0.5B	569	1949	B always small	3/3	1868	–	–
1.5B→0.5B	569	1949	C ridge mid-decode	3/3	1909	–	1.23
1.5B→0.5B	569	1949	C + ordered	3/3	1838	–	0.81
1.5B→0.5B	569	1949	D molt	3/3	1864	–	1.34
1.5B→0.5B	569	1949	D, clear-no-wait	2/3	1949	decode_dst	3.12
1.5B→0.5B	569	1949	D, colocated	3/3	1909	–	1.63
1.5B→0.5B	569	1949	D, del-only	0/3	1951	load_dst	–
1.5B→0.5B	569	1949	big, ignores pressure	0/3	1959	decode_src	–
1.5B→0.5B	818	2198	A restart	3/3	2127	–	1.87
1.5B→0.5B	818	2198	A′ re-prefill	3/3	2176	–	1.49
1.5B→0.5B	818	2198	B always small	3/3	2126	–	–
1.5B→0.5B	818	2198	C ridge mid-decode	3/3	2180	–	2.93
1.5B→0.5B	818	2198	C + ordered	3/3	2093	–	2.43
1.5B→0.5B	818	2198	D molt	3/3	2121	–	1.49
1.5B→0.5B	818	2198	D, clear-no-wait	0/3	2211	decode_dst	–
1.5B→0.5B	818	2198	D, colocated	3/3	2166	–	3.08
1.5B→0.5B	818	2198	D, del-only	0/3	2199	load_dst	–
1.5B→0.5B	818	2198	big, ignores pressure	0/3	2206	decode_src	–

Table 5: Memory schedules. All of them compute the same transfer; they differ only in what is resident when.

schedule	order	resident at peak
colocated	load receiver, map, drop sender	sender + receiver + mapper
del-only	drop sender refs, map, load receiver	sender (cached by MLX) + receiver
clear-no-wait	drop, `clear_cache()`, load at once	depends on reclaim timing
ordered	drop, sync, clear, *wait*, map, drop mapper, load	$\max(\text{sender}, \text{KV} + \text{mapper}, \text{receiver})$